1.2 Relations and KB used in CLUTRR Benchmark

$$
\begin{aligned}
[\texttt{grand}, X, Y] &\vdash [[\texttt{child}, X, Z], [\texttt{child}, Z, Y]],\\
[\texttt{grand}, X, Y] &\vdash [[\texttt{SO}, X, Z], [\texttt{grand}, Z, Y]],\\
[\texttt{grand}, X, Y] &\vdash [[\texttt{grand}, X, Z],\\
&\quad [\texttt{sibling}, Z, Y]],\\
[\texttt{inv-grand}, X, Y] &\vdash [[\texttt{inv-child}, X, Z],\\
&\quad [\texttt{inv-child}, Z, Y]],\\
[\texttt{inv-grand}, X, Y] &\vdash [[\texttt{sibling}, X, Z],\\
&\quad [\texttt{inv-grand}, Z, Y]],\\
[\texttt{child}, X, Y] &\vdash [[\texttt{child}, X, Z],\\
&\quad [\texttt{sibling}, Z, Y]],\\
[\texttt{child}, X, Y] &\vdash [[\texttt{SO}, X, Z],\\
&\quad [\texttt{child}, Z, Y]],\\
[\texttt{inv-child}, X, Y] &\vdash [[\texttt{sibling}, X, Z],\\
&\quad [\texttt{inv-child}, Z, Y]],\\
[\texttt{inv-child}, X, Y] &\vdash [[\texttt{child}, X, Z],\\
&\quad [\texttt{inv-grand}, Z, Y]],\\
[\texttt{sibling}, X, Y] &\vdash [[\texttt{child}, X, Z],\\
&\quad [\texttt{inv-un}, Z, Y]],\\
[\texttt{sibling}, X, Y] &\vdash [[\texttt{inv-child}, X, Z],\\
&\quad [\texttt{child}, Z, Y]],\\
[\texttt{sibling}, X, Y] &\vdash [[\texttt{sibling}, X, Z],\\
&\quad [\texttt{sibling}, Z, Y]],\\
[\texttt{in-law}, X, Y] &\vdash [[\texttt{child}, X, Z],\\
&\quad [\texttt{SO}, Z, Y]],\\
[\texttt{inv-in-law}, X, Y] &\vdash [[\texttt{SO}, X, Z],\\
&\quad [\texttt{inv-child}, Z, Y]],\\
[\texttt{un}, X, Y] &\vdash [[\texttt{sibling}, X, Z],\\
&\quad [\texttt{child}, Z, Y]],\\
[\texttt{inv-un}, X, Y] &\vdash [[\texttt{inv-child}, X, Z],\\
&\quad [\texttt{sibling}, Z, Y]],
\end{aligned}
$$

In the CLUTRR Benchmark, the following kinship relations are used: *son, father, husband, brother, grandson, grandfather, son-in-law, father-in-law, brother-in-law, uncle, nephew, daughter, mother, wife, sister, granddaughter, grandmother, daughter-in-law, mother-in-law, sister-in-law, aunt, niece*.

We used a small, tractable, and logically sound KB of rules as mentioned above. We carefully select this set of deterministic rules to avoid ambiguity in the resolution. We use gender-neutral predicates and resolve the gender of the predicate in the head $\mathcal{H}$ of a clause $\mathcal{C}$ by deducing the gender of the second constant. We have two types of predicates, *vertical* predicates (parent-child relations) and *horizontal* predicates (sibling or significant other). We denote all the vertical predicates by its *child-to-parent* relation and append the prefix `inv-` to the predicates for the corresponding *parent-to-child* relation. For example, `grandfatherOf` is denoted by the gender-neutral predicate $[\texttt{inv-grand}, X, Y]$, where the gender is determined by the gender of Y.

1.3 Effect of placeholder size and split

To analyze whether the language models fail to learn a robust mapping from natural language narratives to underlying logical facts, we re-run the generalization experiments with a reduced placeholder size (20% of the full collected placeholders) *and* we keep the same placeholders for both training and testing. We observe all language-based models are now competitive with respect to GAT on both training regimes $k = 2, 3$ and $k = 2, 3, 4$. This shows the need for separating the placeholder split to effectively test systematic generalization because otherwise, current NLU systems tend to exploit the underlying language layer to arrive at the correct answer.

1.4 More evaluations on Robust Reasoning

We performed several additional experiments to analyze the effect of different training regimes in the Robust Reasoning setup (Table 3) of CLUTRR. Specifically, we want to analyze the effect on zero-shot generalization and robustness when training with different noisy data settings. We notice that the GAT model, having access to the true underlying graph of the puzzles, perform better across different testing scenarios when trained with the noisy data. As the *Supporting facts* contains cycles, it is difficult for GAT to generalize for a dataset with cycles when it is trained on a dataset without cycles. However, when trained with cycles, GAT learns to attend to *all* the paths leading to the correct answer. This effect is disastrous when GAT is tested on *Irrelevant facts* which contains dangling paths as GAT still tries to attend to all the paths. Training on *Irrelevant facts* proved to be most beneficial to GAT, as the model now perfectly attends to *only relevant paths*.

Since *Disconnected facts* contains disconnected paths, the message passing function of the graph is unable to forward any information from the disjoint cliques, thereby having superior testing scores throughout several scenarios.

Experiments on synthetic placeholders. In order to further understand the effect of language placeholders on robustness, we performed another set of experiments where we use bABI (Weston et al., 2015) style simple placeholders (Table 4). We observe a marked increase in performance of all

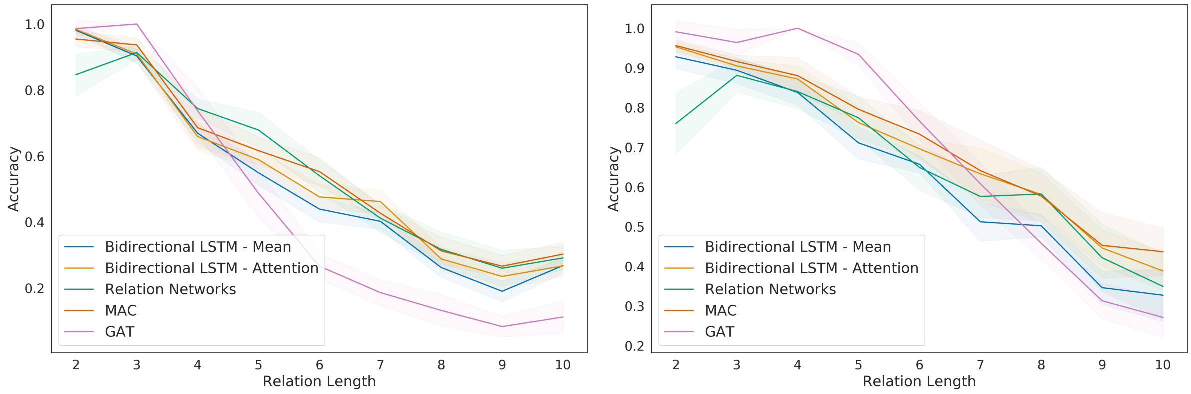

Figure 6: Systematic Generalizability of different models on `CLUTRR-Gen` task (having 20% less placeholders and without training and testing placeholder split), when **Left:** trained with $k = 2$ and $k = 3$ and **Right:** trained with $k = 2, 3$ and 4

Models		Unstructured models (no graph)						Structured model (with graph)
Training	Testing	BiLSTM - Attention	BiLSTM - Mean	RN	MAC	BERT	BERT-LSTM	GAT
Supporting	Clean	0.38 ± 0.04	0.32 ± 0.04	0.4 ± 0.09	0.45 ± 0.03	0.19 ± 0.06	0.39 ± 0.06	**0.92** ± 0.17
	Supporting	0.67 ± 0.06	0.66 ± 0.07	0.68 ± 0.05	0.65 ± 0.04	0.32 ± 0.09	0.57 ± 0.04	**0.98** ± 0.01
	Irrelevant	0.44 ± 0.03	0.39 ± 0.03	**0.51** ± 0.08	0.46 ± 0.09	0.2 ± 0.06	0.36 ± 0.05	0.5 ± 0.23
	Disconnected	0.31 ± 0.21	0.25 ± 0.16	0.47 ± 0.08	0.41 ± 0.06	0.2 ± 0.08	0.32 ± 0.04	**0.92** ± 0.05
Irrelevant	Clean	0.57 ± 0.05	0.56 ± 0.05	0.46 ± 0.13	0.67 ± 0.05	0.24 ± 0.06	0.46 ± 0.08	**0.92** ± 0.0
	Supporting	0.38 ± 0.22	0.31 ± 0.16	0.61 ± 0.07	0.61 ± 0.04	0.27 ± 0.06	0.46 ± 0.04	**0.77** ± 0.12
	Irrelevant	0.51 ± 0.06	0.52 ± 0.06	0.5 ± 0.04	0.56 ± 0.04	0.25 ± 0.06	0.53 ± 0.06	**0.93** ± 0.01
	Disconnected	0.44 ± 0.26	0.54 ± 0.27	0.55 ± 0.05	0.61 ± 0.06	0.26 ± 0.03	0.45 ± 0.08	**0.85** ± 0.25
Disconnected	Clean	0.45 ± 0.02	0.47 ± 0.03	0.53 ± 0.09	0.5 ± 0.06	0.22 ± 0.09	0.44 ± 0.05	**0.75** ± 0.07
	Supporting	0.47 ± 0.03	0.46 ± 0.05	0.54 ± 0.03	0.58 ± 0.06	0.22 ± 0.06	0.38 ± 0.08	**0.78** ± 0.12
	Irrelevant	0.47 ± 0.05	0.48 ± 0.03	0.52 ± 0.04	0.51 ± 0.05	0.17 ± 0.04	0.38 ± 0.05	**0.56** ± 0.26
	Disconnected	0.57 ± 0.07	0.57 ± 0.06	0.45 ± 0.11	0.4 ± 0.1	0.17 ± 0.05	0.47 ± 0.06	**0.96** ± 0.01
Average		0.47 ± 0.08	0.46 ± 0.08	0.52 ± 0.07	**0.53** ± 0.06	0.23 ± 0.07	0.43 ± 0.05	**0.82** ± 0.09

Table 3: Testing the robustness of the various models when trained various types of noisy facts and evaluated on other noisy / clean facts. The types of noise facts (supporting, irrelevant and disconnected) are defined in Section 3.5 of the main paper.

Models		Unstructured models (no graph)						Structured model (with graph)
Training	Testing	BiLSTM - Attention	BiLSTM - Mean	RN	MAC	BERT	BERT-LSTM	GAT
Supporting	Clean	0.96 ± 0.01	**0.97** ± 0.01	0.88 ± 0.05	0.94 ± 0.02	0.48 ± 0.08	0.57 ± 0.08	0.92 ± 0.17
	Supporting	0.96 ± 0.03	0.96 ± 0.03	0.97 ± 0.01	0.97 ± 0.01	0.75 ± 0.07	0.88 ± 0.05	**0.98** ± 0.01
	Irrelevant	0.92 ± 0.02	**0.93** ± 0.01	0.9 ± 0.03	0.91 ± 0.01	0.56 ± 0.04	0.54 ± 0.06	0.5 ± 0.23
	Disconnected	0.8 ± 0.04	0.83 ± 0.04	0.76 ± 0.08	0.86 ± 0.04	0.27 ± 0.06	0.42 ± 0.08	**0.92** ± 0.05
Irrelevant	Clean	0.63 ± 0.02	0.61 ± 0.07	0.85 ± 0.09	0.8 ± 0.07	0.53 ± 0.09	0.44 ± 0.06	**0.92** ± 0.0
	Supporting	0.66 ± 0.03	0.64 ± 0.04	0.69 ± 0.06	0.76 ± 0.06	0.42 ± 0.08	0.43 ± 0.08	**0.77** ± 0.12
	Irrelevant	0.89 ± 0.04	0.86 ± 0.1	0.74 ± 0.11	0.78 ± 0.06	0.61 ± 0.1	0.83 ± 0.06	**0.93** ± 0.01
	Disconnected	0.64 ± 0.02	0.62 ± 0.05	0.72 ± 0.05	0.73 ± 0.04	0.41 ± 0.04	0.61 ± 0.05	**0.85** ± 0.25
Disconnected	Clean	0.9 ± 0.05	0.82 ± 0.12	**0.94** ± 0.02	0.93 ± 0.04	0.68 ± 0.07	0.64 ± 0.02	0.75 ± 0.07
	Supporting	0.87 ± 0.04	0.82 ± 0.05	0.85 ± 0.03	**0.88** ± 0.04	0.54 ± 0.08	0.5 ± 0.05	0.78 ± 0.12
	Irrelevant	**0.87** ± 0.03	0.85 ± 0.03	0.83 ± 0.03	0.87 ± 0.02	0.59 ± 0.09	0.58 ± 0.09	0.56 ± 0.26
	Disconnected	0.91 ± 0.04	0.91 ± 0.03	0.8 ± 0.17	0.71 ± 0.11	0.49 ± 0.1	0.79 ± 0.1	**0.96** ± 0.01
Average		0.83 ± 0.08	0.82 ± 0.08	0.83 ± 0.07	**0.84** ± 0.06	0.58 ± 0.07	0.60 ± 0.05	**0.82** ± 0.09

Table 4: Testing the robustness on toy placeholders of the various models when trained various types of noisy facts and evaluated on other noisy / clean facts. The types of noise facts (supporting, irrelevant and disconnected) are defined in Section 3.5 of the main paper.

NLU models, where they significantly decrease the gap between their performance with that of GAT, even outperforming GAT on various settings. This shows the significance of using paraphrased placeholders in devising the complexity of the dataset.

1.5 Comparison among different entity embedding policies

In Cloze style reading comprehension tasks, it is sometimes customary to choose UNK embeddings for entity placeholders. (Chen et al., 2016) In our task, however, choosing UNK embeddings for entities is not feasible as the query involves two entities themselves. During preprocessing of our dataset, we convert the entity names into a Cloze-style setup where each entity is replaced by *@entity-n* token. However, one has to be careful not to assign tokens in the same order for all the stories, which will lead to obvious overfitting since the models will learn to work around positional markers as shown in Chen et al. (2016). Therefore, we randomize the Cloze-style entities themselves for each story. We experimented with three different policies of choosing the entity embeddings:

1. *Fixed Random Embeddings*: One simple and intuitive choice is to assign a random embedding to each entity and keep it fixed throughout the training. During our data-processing pipeline, we ensure that all the entity tokens are randomized using a pool of entity tokens, hence the chances of a model learning to exploit the positional markers are slim.

2. *Randomized Random Embeddings*: We can go one step further and randomize the random embeddings at *each epoch*. This aggressive strategy does not let the model learn any positional markings at all, however it might hamper the learning ability of models as the entity representations are changing arbitrarily.

3. *Learned Random Embeddings*: Since our data pre-processing pipeline randomly assigns the entities on each story, we can as well learn a pool of n entities, from which a subset is always used to replace the entities.

We chose to report all experiments with respect to fixed random embeddings. We compared different embedding policies with respect to the Systematic Generalization task. We show a comparison between the Bidirectional LSTM and GAT in Figure 7. We see that the fixed embedding policy has better Systematic Generalization score, although the advantage is minor compared to the other schemes. For GAT, the advantage is practically nil for the different schemes which shows that a Graph Neural Network performs inductive reasoning in the same manner irrespective of the initial node embedding representation.

1.6 AMT Data collection process

We use ParlAI (Miller et al., 2017) Mturk interface to collect paraphrases from the users. Specifically, given a set of facts, we ask the users to paraphrase the facts into a story. The users (*turkers*) are free to construct any story they like as long as they mention all the entities and all the relations among them. We also provide the head $\mathcal{H}$ of the clause as an *inferred* relation and specifically instruct the users to *not* mention it in the paraphrased story. In order to evaluate the paraphrased stories, we ask the turkers to peer review a story paraphrased by a different turker. Since there are two tasks - paraphrasing a story and rating a story - we choose to pay 0.5$ for each annotation. A sample task description in our MTurk interface is as follows:

> In this task, you will need to write a short, simple story based on a few facts. **It is crucial that the story mentions each of the given facts at least once.** The story does not need to be complicated! It just needs to be grammatical and mention the required facts.
>
> After writing the story, you will be asked to evaluate the quality of a generated story (based on a different set of facts). **It is crucial that you check whether the generated story mentions each of the required facts.**
>
> *Example of good and bad stories: Good Example*
>
> **Facts to Mention**
>
> - John is the father of Sylvia.
> - Sylvia has a brother Patrick.
>
> **Implied Fact**: John is the father of Patrick.
>
> **Written story**
>
> John is the proud father of the lovely Sylvia. Sylvia has a love-hate relationship with her brother Patrick.
>
> *Bad Example*
>
> **Facts to Mention**
>
> - Vincent is the son of Tim.
> - Martha is the wife of Tim.
>
> **Implied Fact** : Martha is Vincent's mother.
>
> **Written story**
>
> Vincent is married at Tim and his mother is Martha.
>
> *The reason the above story is bad*:
>
> - This story is bad because it is nonsense / ungrammatical.
> - This story is bad because it does not mention the proper facts.
> - This story is bad because it reveals the implied fact.